บทที่ 13

การดีบั๊ก การสร้างแอปพลิเคชัน และการเผยแพร่ด้วย EAS Build

การพัฒนาโมบายแอปพลิเคชันจนสามารถพิมพ์โค้ดรันได้ในเครื่องจำลอง (Simulator) ถือเป็นเพียงครึ่งแรกของกระบวนการพัฒนา ด่านสำคัญถัดไปคือการแกะรอยแก้ไขข้อผิดพลาดในระดับอุปกรณ์จริง (Native Debugging) และการแปลงซอร์สโค้ด React Native ให้กลายเป็นไฟล์ติดตั้งไบนารีเนทีฟ (เช่น `.apk` สำหรับ Android หรือ `.ipa` สำหรับ iOS) เพื่อส่งขึ้น Store ในบทเรียนสุดท้ายนี้ เราจะเรียนรู้เทคนิคการดีบั๊กใน Expo SDK 54, ทำความเข้าใจระบบคลาวด์คอมไพล์แอป **EAS Build** และขั้นตอนเตรียมความพร้อมเพื่อเผยแพร่แอปพลิเคชันสู่ผู้ใช้งานจริงทั่วโลก

13.1 เทคนิคการตรวจสอบและแก้ไขข้อผิดพลาดใน Expo SDK 54 (Debugging)

ในการเขียนแอปพลิเคชัน ย่อมเกิดข้อผิดพลาดหรือข้อขัดข้องขึ้นได้ ทั้งจากโค้ดจาวาสคริปต์ของเราเอง หรือเออเรอร์ระดับเครื่องยนต์เนทีฟ (Native Crash) Expo SDK 54 ได้จัดสรรเครื่องมือในการค้นหาความจริง (Expo, 2024) ดังนี้:

1. การเรียกใช้งาน Expo Developer Menu

ขณะรันแอปบนจำลองหรือสมาร์ทโฟนจริงผ่าน Expo Go เราสามารถกดเปิดเมนูผู้พัฒนาได้โดย:

- **ในเครื่อง Simulator:** กดปุ่ม `m` บนคีย์บอร์ดในเทอร์มินัลที่รัน Expo CLI
- **ในสมาร์ทโฟนจริง:** ทำการ **เขย่าโทรศัพท์มือถือ (Shake Device)**

ในเมนูนี้เราสามารถสั่งรีโหลดแอปใหม่ทั้งหมด (Reload) หรือเปิดใช้งานเครื่องมือการสืบค้นองค์ประกอบ UI (Toggle Inspector)

2. การเชื่อมต่อ Chrome Developer Tools

เราสามารถดีบั๊กโค้ด JavaScript ที่กำลังรันอยู่บนโทรศัพท์มือถือผ่านเครื่องมืออันคุ้นเคยของ Google Chrome โดยเปิดบราวเซอร์ Chrome ขึ้นมาแล้วทำตามขั้นตอนดังนี้:

1. เปิด Developer Tools ใน Chrome (กด `F12`)
2. กดปุ่ม `Ctrl + Shift + P` (หรือ `Cmd + Shift + P` บน Mac) พิมพ์คำสั่ง `Open Debugger` หรือคลิกที่ลิงก์ดีบั๊กเกอร์ที่ Expo CLI แสดงขึ้นในหน้าต่างเทอร์มินัล (มักเป็นคีย์ลัด `j` ในคอมพิวเตอร์)

3. ระบบจะสลับหน้าจอไปที่แถบดีบั๊กเกอร์ ซึ่งเปิดโอกาสให้เราสามารถดักจับตัวแปรในหน้า Console, ใส่จุดหยุดทำงาน (Breakpoints) และตรวจสอบตาราง Call Stack ได้ทันทีเสมือนการพัฒนาเว็บแอปพลิเคชัน

13.2 สถาปัตยกรรมและหลักการของ Expo Application Services (EAS)

ในอดีต การสร้างไฟล์ติดตั้ง (Build App) ของ React Native เป็นเรื่องยุ่งยากอย่างมหาศาล เพราะนักพัฒนาจำเป็นต้องตั้งค่าแอนดรอยด์ SDK อย่างหนาแน่นในเครื่อง หรือหากต้องการสร้างไฟล์สำหรับ iOS ก็ถูกบังคับให้ต้องมีเครื่องคอมพิวเตอร์ Mac ที่ลงระบบ Xcode เท่านั้น

เพื่อขจัดความยุ่งยากนี้ Expo จึงได้พัฒนาบริการคลาวด์สำเร็จรูปที่ชื่อว่า **EAS (Expo Application Services)** (Expo, 2024) ซึ่งเป็นระบบเซิร์ฟเวอร์คลาวด์ประสิทธิภาพสูงที่จะคอยรับโค้ดของเราขึ้นไปคอมไพล์เป็นไฟล์ติดตั้งเนทีฟให้โดยอัตโนมัติ

ข้อดีของ EAS Build:

• **ไม่ต้องมีเครื่อง Mac ก็สร้างแอป iOS ได้:** เพราะการ Build เกิดขึ้นบนเซิร์ฟเวอร์คลาวด์ของ Expo เอง

• **ประหยัดทรัพยากรเครื่องคอมพิวเตอร์:** ไม่ต้องรันคอมไพล์หนักหน่วงจน RAM เต็มในเครื่องโลคัลของเรา

• **จัดการคีย์ความปลอดภัยให้อัตโนมัติ:** คอยดูแลและสร้างใบรับรองดิจิทัล (Signing Credentials) สำหรับแอนดรอยด์และ iOS ให้ปลอดภัย

13.3 การจัดเตรียมความพร้อมแอปพลิเคชันใน `app.json`

ก่อนที่เราจะส่งโค้ดขึ้นไปประกอบเป็นไฟล์แอปพลิเคชันบนคลาวด์ เราต้องระบุคุณลักษณะที่จำเป็นในไฟล์ `app.json` ซึ่งทำหน้าที่เสมือนบัตรประชาชนระบุตัวตนของแอปพลิเคชัน:

```
{
 "expo": {
  "name": "My Personal Diary",          // ชื่อแอปที่จะโชว์ใต้ไอคอนบนหน้าจอโทรศัพท์
  "slug": "my-personal-diary",
  "version": "1.0.0",                   // เลขเวอร์ชันของแอปที่โชว์ให้ผู้ใช้เห็น
  "orientation": "portrait",
  "icon": "./assets/icon.png",          // ที่อยู่รูปภาพไอคอนแอปพลิเคชัน
  "userInterfaceStyle": "dark",
  "splash": {
   "image": "./assets/splash-image.png", // หน้าจอต้อนรับตอนเปิดแอป (Splash Screen)
   "resizeMode": "contain",
```

```
    "backgroundColor": "#0f172a"
     },
   "ios": {
    "supportsTablet": true,
    "bundleIdentifier": "com.yourdomain.mydiary", // รหัสระบุตัวตนห้ามซ้ำของ iOS
    "buildNumber": "1"
     },
   "android": {
    "adaptiveIcon": {
     "foregroundImage": "./assets/adaptive-icon.png",
     "backgroundColor": "#0f172a"
       },
    "package": "com.yourdomain.mydiary",          // รหัสระบุตัวตนห้ามซ้ำของ Android
(Package Name)
    "versionCode": 1                  // เลขรันของรุ่นแอปในสารระบบดาต้าเบสสโตร์ (ต้องเพิ่มค่าขึ้นทีละ 1 ทุก
ครั้งที่อัปเดตสโตร์)
     }
   }
}
```

คำเตือน (Warning): ฟิลด์ `android.package` และ `ios.bundleIdentifier` ต้องใช้รูปแบบจดโดเมนย้อนกลับ (Reverse Domain Name) เช่น `com.`บริษัท.ชื่อแอป และชื่อนี้ห้ามไปซ้ำกับแอปพลิเคชันอื่นใดในโลกเด็ดขาด มิฉะนั้นสโตร์จะปฏิเสธการอัปโหลดเข้าสู่คลังระบบ

13.4 การตั้งค่าโปรไฟล์การคอมไพล์ใน `eas.json`

เราสามารถกำหนดค่าพฤติกรรมการคอมไพล์แอปแยกประเภทได้ผ่านไฟล์ `eas.json` (ซึ่งตั้งอยู่ที่โฟลเดอร์นอกสุดของโปรเจกต์) โดยเรามักสร้างไฟล์นี้อัตโนมัติด้วยคำสั่ง:

```
eas build:configure
```

ตัวอย่างการกำหนดค่าในไฟล์ `eas.json`:

สำหรับการนำมาสอนหรือทดสอบติดตั้งเองในเครื่อง โดยไม่ต้องส่งขึ้นสโตร์ทันที เราจะตั้งค่าโปรไฟล์สำหรับดาวน์โหลดไฟล์ APK (สำหรับ Android) เอาไว้ทดลองรันภายในองค์กร (Internal Distribution) ดังนี้:

```
{
 "cli": {
  "version": ">= 10.0.0"
  },
```

```
"build": {
  "development": {
    "developmentClient": true,
    "distribution": "internal"
     },
  "preview": {
    "distribution": "internal",
    "android": {
      "buildType": "apk"          // บังคับให้คอมไพล์ออกมาเป็นไฟล์ติดตั้งด่วนประเภท .apk
        }
     },
  "production": {
    "distribution": "store"        // บังคับเตรียมไฟล์สำหรับส่งขึ้น App Store / Play Store
     }
  }
}
```

13.5 ขั้นตอนการสั่งรันคอมไพล์ด้วยคำสั่ง EAS CLI

เมื่อเตรียมความพร้อมไฟล์คอนฟิกสำเร็จแล้ว ให้ทำตามขั้นตอนการยิงคำสั่งคอมไพล์จริงบนคลาวด์ด้วย EAS CLI (Expo, 2024) ดังนี้:

ขั้นตอนที่ 1: ติดตั้ง EAS CLI ส่วนกลางในเครื่องคอมพิวเตอร์ของคุณ

```
npm install -g eas-cli
```

ขั้นตอนที่ 2: ลงชื่อเข้าใช้บัญชี Expo

ทำการล็อกอินระบบด้วยบัญชีที่สมัครไว้กับเว็บไซต์ Expo:

```
eas login
```

ขั้นตอนที่ 3: สั่งรันเริ่มคอมไพล์ภาพแอป

ส่งคำสั่งสั่งรันโปรไฟล์ `preview` เพื่อสร้างไฟล์ติดตั้ง APK สำหรับ Android:

```
eas build --platform android --profile preview
```

ขั้นตอนที่เกิดขึ้นเบื้องหลัง (Build Pipeline):

1. EAS CLI จะตรวจสอบความถูกต้องของไฟล์ `app.json` และ `eas.json`

2. ระบบจะบีบอัดแพ็คเกจซอร์สโค้ดฝั่ง Client (เฉพาะโค้ดดิ้ง ไม่รวมโฟลเดอร์ node_modules) ส่งผ่านสายขึ้นคลาวด์ของ Expo

3. คลาวด์ของ Expo จะเปิดระบบจำลองเซิร์ฟเวอร์ขึ้นมาทำการติดตั้งแพ็คเกจและทำลายคอมไพล์โค้ดเป็นไบนารีระดับเนทีฟ

4. เมื่อกระบวนการเสร็จสิ้น เทอร์มินัลของคุณจะแสดง **ลิงก์ดาวน์โหลดเว็บ** พร้อมกับรหัส QR Code ขึ้นมาบนหน้าจอคอมพิวเตอร์

5. ผู้เรียนสามารถนำกล้องถ่ายรูปของสมาร์ทโฟนแอนดรอยด์สแกน QR Code นี้เพื่อเปิดดาวน์โหลดไฟล์ติดตั้ง .apk มาติดตั้งรันทำงานบนโทรศัพท์มือถือจริงได้ทันที!

13.6 แนวทางและกระบวนการเผยแพร่แอปขึ้น App Stores

เมื่อผู้พัฒนาทดสอบแอปจนมั่นใจและพร้อมที่จะเผยแพร่ให้คนดาวน์โหลดใช้งานทั่วไป ขั้นตอนการดำเนินงานขึ้นร้านค้าทางการมีดังนี้:

```
graph TD
    A[คอมไพล์แอปพลิเคชันแบบ Production] --> B{ระบบปฏิบัติการ?}
    B -->|Android| C[สร้างไฟล์ .aab - Android App Bundle]
    B -->|iOS| D[สร้างไฟล์ .ipa พร้อมใบรับรอง Apple]
    C --> E[สมัครบัญชี Google Play Console - มีค่าบริการครั้งเดียว $25]
    D --> F[สมัครบัญชี Apple Developer Program - ค่าบริการรายปี $99]
    E --> G[อัปโหลดไฟล์ผ่านเว็บ Google Play Console / eas submit]
    F --> H[อัปโหลดไฟล์ผ่าน Transporter หรือ eas submit]
    G --> I[ตั้งค่าฟอร์มความปลอดภัย / ข้อมูลจำกัดอายุ]
    H --> J[ตั้งค่า App Store Connect / นโยบายความเป็นส่วนตัว]
    I --> K[ส่งตรวจสอบและได้รับอนุมัติขึ้นดาวน์โหลดบน Play Store]
    J --> L[ส่งตรวจสอบและได้รับอนุมัติขึ้นดาวน์โหลดบน App Store]
```

การส่งด้วยความสามารถคำสั่งอัตโนมัติ (EAS Submit):

หลังจากที่สร้างแอปแบบโปรไฟล์การผลิตเสร็จสิ้น เราสามารถสั่งส่งอัปโหลดตรงเข้าร้านค้าสโตร์ต่างๆ ผ่านคำสั่ง:

```
eas submit --platform android
```

คำสั่งนี้จะดึง Credentials และเชื่อม API ไดเรกทอรีของสโตร์เพื่อส่งมอบแอปขึ้นไปเตรียมรีวิวในฝั่งหลังบ้านได้อย่างรวดเร็วและปลอดภัยลดความซับซ้อนของผู้พัฒนาแอปพลิเคชันอย่างมหาศาล

13.7 ขั้นตอนการอัปเกรดเวอร์ชันของ Expo SDK (Upgrading Expo SDK)

ทีมงาน Expo จะทำการเปิดตัวรุ่น SDK ใหม่ ๆ (Major Releases) ทุก ๆ 3 เดือนเพื่อเพิ่มฟีเจอร์ ปรับแต่งประสิทธิภาพ และอัปเกรดรุ่นซอฟต์แวร์เบื้องหลังของ React Native และ React ให้ทันสมัยและปลอดภัย การมีทักษะในการอัปเกรด SDK ข้ามเวอร์ชันจึงเป็นเรื่องจำเป็นอย่างยิ่งสำหรับนักพัฒนาระดับอาชีพ โดยมีลำดับขั้นตอนที่เป็นมาตรฐานดังนี้:

ขั้นตอนที่ 1: ตรวจสอบและอัปเดตเครื่องมือ EAS CLI

ก่อนเริ่มทำการอัปเกรดโปรเจกต์ ควรทำการปรับปรุงเครื่องมือบิลด์ให้เป็นเวอร์ชันล่าสุดเพื่อรองรับพารามิเตอร์ของ SDK ตัวใหม่:

```
npm install -g eas-cli
```

ขั้นตอนที่ 2: ติดตั้งรุ่นของตัวหลัก Expo SDK ปลายทาง

เราสามารถระบุให้โปรเจกต์เปลี่ยนไปใช้ Expo เวอร์ชันล่าสุดได้ด้วยคำสั่ง:

```
npm install expo@latest
```

หรือหากต้องการเจาะจงระบุเป็นรุ่นเวอร์ชันเฉพาะ เช่น ขยับขึ้นไปที่ Expo SDK 54 ให้ป้อนระบุรุ่นดังนี้:

```
npx expo install expo@^54.0.0
```

ขั้นตอนที่ 3: จัดการปรับปรุงรุ่นไลบรารีอื่น ๆ (Dependency Reconciliation)

ข้อควรจำ (Important): นี่คือขั้นตอนที่สำคัญที่สุดในการอัปเกรดระบบ Expo

เมื่อตัวแกนหลักอย่าง `expo` ได้ขยับรุ่นขึ้นไปแล้ว แพ็คเกจอื่น ๆ เช่น `react-native`, `react`, `expo-router` หรือ `expo-image` อาจจะยังทำงานค้างอยู่ที่เวอร์ชันเก่าซึ่งจะทำให้รันแอปพังหรือบิลด์ไม่ผ่าน

เราห้ามลบและไล่ติดตั้งทีละตัวด้วยตัวเอง แต่ให้พิมพ์รันคำสั่งพิเศษของ Expo ดังนี้:

``bash npx expo install --fix ` คำสั่งนี้จะทำการสแกนไฟล์ package.json` ค้นหาความขัดแย้งของรุ่นทั้งหมด และทำการสั่งดาวน์โหลดแพ็คเกจอื่น ๆ ทุกตัวที่เกี่ยวข้องให้เข้าสู่รุ่นเวอร์ชันที่เหมาะสมและผ่านการรับรองความเสถียร (Compatible versions) จากทางทีมพัฒนาของ Expo โดยออโต้

ขั้นตอนที่ 4: ล้างหน่วยความจำแคช (Clear Metro Cache)

ทุกครั้งที่ขยับรุ่นซอฟต์แวร์ เราต้องบังคับล้างประวัติแคชของการรันคอมไพเลอร์ Metro เพื่อไม่ให้มีเศษไฟล์เก่ามาทำให้แอปทำงานพังเงียบ:

```
npx expo start -c
```

(คีย์ออปชัน `-c` ย่อมาจากคำสั่ง `--clear` เพื่อล้างแคชระบบก่อนสตาร์ตโปรแกรมรันใหม่)

ขั้นตอนที่ 5: บิลด์ Dev Client ใหม่ (กรณีใช้งานคัสตอมไบนารี)

หากแอปพลิเคชันเดิมทำระบบใช้งานร่วมกับ Custom Development Clients (แทนการใช้แอป Expo Go มาตรฐาน) การขยับ SDK จะมีผลทำให้โค้ดฝั่งเนทีฟ (Android/iOS) ขยับรุ่นตามไปด้วย นักพัฒนาจำเป็นต้องสั่งรัน `eas build --profile development` เพื่อรับไฟล์ติดตั้งตัวทดสอบใหม่มาลงทับในสมาร์ทโฟนเสมอ

สรุปท้ายบทเรียน

เทคนิคการตรวจสอบและแก้ไขข้อผิดพลาดใน Expo SDK 54: กระบวนการตรวจหาและดีบั๊กข้อผิดพลาดใน Expo SDK 54 สามารถใช้ Expo Developer Menu ในการสั่งรีโหลดหน้าจอหรือเปิดระบบ Inspector โดยสามารถเรียกใช้งานผ่านคีย์บอร์ด m ในเครื่องจำลองหรือการเขย่าอุปกรณ์จริง และสามารถเชื่อมโยงคำสั่ง JavaScript เข้ากับ Chrome Developer Tools ผ่านการทริกเกอร์ j ในคอมพิวเตอร์เพื่อเปิดหน้าต่างตรวจสอบ Console บันทึกสเตตัสและใส่จุดหยุดทำงานได้อย่างละเอียด

สถาปัตยกรรมและหลักการของ Expo Application Services (EAS): บริการ EAS Build คลาวด์อำนวยความสะดวกในการจัดทำไฟล์ติดตั้งเนทีฟสำเร็จรูปให้ผู้พัฒนาโดยแก้ปัญหาความยุ่งยากของการจัดตั้งเครื่อง Android SDK หรือความจำเป็นต้องมีเครื่อง Mac ในการคอมไพล์ระบบ iOS โดยระบบจะนำซอร์สโค้ดไปประกอบและจัดการคีย์ความปลอดภัยของแอปพลิเคชัน (Signing Credentials) บนคลาวด์เซิร์ฟเวอร์ความเร็วสูงให้โดยอัตโนมัติ

การจัดเตรียมความพร้อมแอปพลิเคชันใน app.json: ก่อนส่งมอบโค้ดไปประกอบไฟล์ติดตั้งระดับเนทีฟ ผู้พัฒนาจำเป็นต้องระบุคุณสมบัติตัวตนของแอปพลิเคชันในไฟล์ app.json เช่น ชื่อแอป เวอร์ชันของรุ่น ที่อยู่ของรูปไอคอนและภาพต้อนรับตอนเริ่มแอป รวมถึงการตั้งรหัสผ่านชุดจดทะเบียนที่ห้ามซ้ำกันเด็ดขาดในระดับสากลอย่าง android.package และ ios.bundleIdentifier

การตั้งค่าโปรไฟล์การคอมไพล์ใน eas.json: ไฟล์ eas.json ใช้สำหรับระบุเป้าหมายและขั้นตอนพฤติกรรมการบิลด์แอปพลิเคชัน โดยเราเขียนปรับแต่งให้แยกประเภทโปรไฟล์ได้ เช่น โปรไฟล์ development เพื่อสร้างไคลเอนต์ทดสอบในทีม, โปรไฟล์ preview ที่บังคับคอมไพล์ออกมา

เป็นไฟล์ติดตั้ง .apk โดยตรงสำหรับเครื่อง Android และโปรไฟล์ production เพื่อสร้างไฟล์ติดตั้งมาตรฐานในการส่งตัวแอปเตรียมยื่นขึ้นร้านค้าอย่างเป็นทางการ

ขั้นตอนการสั่งรันคอมไพล์ด้วยคำสั่ง EAS CLI: ลำดับการสั่งประกอบแอปเริ่มจากติดตั้ง eas-cli และทำรายการล็อกอินด้วยคำสั่ง eas login จากนั้นจึงยิงคำสั่งคอมไพล์หลักตามโปรไฟล์ที่ตั้งค่าไว้ ซึ่งระบบจะนำรหัสโค้ดฝั่งไคลเอนต์ขึ้นประมวลผลบนเซิร์ฟเวอร์คลาวด์ของ Expo และแสดงที่อยู่ลิงก์และ QR Code บนหน้าต่างเทอร์มินัลเพื่อให้ผู้พัฒนาดาวน์โหลดไฟล์ไบนารี .apk มาลงตรวจสอบความเรียบร้อยบนโทรศัพท์จริงได้

แนวทางและกระบวนการเผยแพร่แอปขึ้น App Stores: กระบวนการยื่นแอปขึ้นร้านค้าทำได้โดยบิลด์โปรไฟล์การผลิตเพื่อให้ได้ไฟล์ .aab สำหรับ Android หรือไฟล์ .ipa สำหรับ iOS เพื่อเตรียมยื่นขอขึ้นร้านค้าอย่างเป็นทางการผ่านบัญชี Google Play Console หรือ Apple Developer Program โดยใช้เครื่องมือส่งเอกสารผ่านเว็บบอร์ดหรือสั่งงานอย่างสะดวกผ่านระบบจัดส่งอัตโนมัติด้วยคำสั่ง eas submit

ขั้นตอนการอัปเกรดเวอร์ชันของ Expo SDK: การปรับแต่งและขยับรุ่นโปรเจกต์ให้มีประสิทธิภาพและมีความปลอดภัยสูงสุดตามรุ่น SDK ล่าสุด ทำได้โดยติดตั้งตัวแกนหลัก expo รุ่นใหม่ จากนั้นสั่งทำงาน npx expo install --fix เพื่อปรับปรุงรุ่นไลบรารีอื่น ๆ ทั้งระบบให้มีความเข้ากันและผ่านการรับรองความเสถียร พร้อมทั้งบังคับล้างประวัติแคชรัน Metro ด้วย npx expo start -c เพื่อเคลียร์ไฟล์เก่าไม่ให้ระบบทำงานขัดข้อง

แบบฝึกหัดท้ายบทเรียน

แบบทดสอบนี้มีวัตถุประสงค์เพื่อประเมินความรู้และความเข้าใจเกี่ยวกับกระบวนการตรวจสอบและดีบั๊กโค้ดแอปพลิเคชัน, สถาปัตยกรรมคลาวด์คอมไพล์ของบริการ Expo Application Services (EAS), การตั้งค่าคุณสมบัติแอปพลิเคชันในไฟล์ `app.json` และการจัดการตั้งค่าโปรไฟล์ใน `eas.json` ตลอดจนกลไกการอัปเดตอัปเกรดเวอร์ชันของ Expo SDK และการนำส่งแอปขึ้นเผยแพร่บนร้านค้าสากล

คำถามประจำบทเรียน (10 ข้อ)

ข้อ 1: หากกำลังเชื่อมต่อแอปพลิเคชันเพื่อทดสอบอยู่บนสมาร์ทโฟนเครื่องจริง และผู้สอนสั่งให้ทำการตรวจสอบหน้าจอ UI ผ่านคำสั่งใน "Expo Developer Menu" จะต้องทริกเกอร์เรียกแผงคำสั่งนี้ขึ้นมาอย่างไรบนหน้าจอโทรศัพท์?

ก. ทำการแตะหน้าจอรัว ๆ 10 ครั้งติดต่อกัน

ข. ทำการเขย่าตัวเครื่องโทรศัพท์มือถือ (Shake Device)

ค. กดปุ่มลดระดับเสียงของเครื่อง 3 ครั้ง

ง. สั่งตัดสัญญาณอินเทอร์เน็ตบนตัวเครื่องชั่วคราว

ข้อ 2: การเข้าถึงหน้าจอดีบั๊กเกอร์สำหรับตรวจเช็คการทำงานของรหัสจาวาสคริปต์ ดักตรวจสอบตัวแปรในแท็บ Console และการปักหมุดจุดหยุดทำงาน (Breakpoints) นักพัฒนาสามารถเปิดหน้าต่างนี้ผ่านเครื่องมือใด?

ก. หน้าจอ Android Studio Logcat

ข. Google Chrome Developer Tools (โดยกดเรียกดีบั๊กเกอร์ผ่าน CLI)

ค. ระบบสไลด์เมนูของตัวจำลอง Simulator

ง. แอปพลิเคชันตั้งค่าพื้นฐานของตัวเครื่องมือถือ

ข้อ 3: คำว่า "EAS" ในระบบบริการคลาวด์ของ Expo ย่อมาจากคำเต็มในข้อใด?

ก. Expo Application Services

ข. Easy App Solutions

ค. Expo Android Systems

ง. Enterprise App Server

ข้อ 4: ข้อใดคือประโยชน์สูงสุดและเห็นภาพชัดเจนที่สุดในการเลือกใช้งานบริการระบบคลาวด์ EAS Build เข้ามาจัดสร้างประกอบไฟล์ติดตั้งแอปพลิเคชัน?

ก. ช่วยเร่งความเร็วในการดาวน์โหลดไฟล์รูปภาพในแอปพลิเคชันของฝั่งลูกค้า

ข. เปิดโอกาสให้นักพัฒนาสามารถสร้างคอมไพล์ไฟล์แอปเนทีฟ (เช่น แอป iOS) ได้สำเร็จจากระยะไกลโดยไม่จำเป็นต้องติดตั้งสภาพแวดล้อม Xcode หรือมีคอมพิวเตอร์ Mac เป็นของตนเอง

ค. ทำหน้าที่เขียนคำสั่งสืบค้น SQL-like ส่งไปยัง Supabase อัตโนมัติ

ง. สั่งปิดสัญญาณสัญญานเครือข่ายมือถือเพื่อประหยัดหน่วยความจำ RAM

ข้อ 5: แฟ้มข้อมูลใดในโฟลเดอร์โครงการ Expo ที่ทำหน้าที่บรรจุรายละเอียดระบุตัวตน ชื่อแอป ไอคอนแอป หน้า Splash Screen และรหัสแพ็คเกจปลายทางของตัวแอปพลิเคชัน?

ก. `eas.json`

ข. `package.json`

ค. `app.json`

ง. `babel.config.js`

ข้อ 6: การกรอกฟิลด์ระบุตัวตนสากลของแอปใน `app.json` เช่น `android.package` หรือ `ios.bundleIdentifier` ข้อใดคือรูปแบบการเขียนชื่อที่ถูกต้องและเป็นมาตรฐานยอมรับในทุกสโตร์?

ก. เขียนข้อความสตริงเพลน ๆ ตัวพิมพ์ใหญ่ เช่น `MY_DIARY_APP`

ข. เขียนเป็นรหัสพิกัดพิกสเปซ เช่น `/app/mydiary`

ค. เขียนโดยใช้รูปแบบโดเมนกลับหัว (Reverse Domain Name Notation) เช่น `com.yourcompany.appname`

ง. เขียนเป็นตัวเลขแสดงลำดับไอดีโครงการ เช่น 123.456.789

ข้อ 7: ในกรณีที่ต้องการเปลี่ยนประเภทของการคอมไพล์สำหรับฝั่ง Android ให้เปลี่ยนจากไฟล์ App Bundle มาเป็นไฟล์ดาวน์โหลดติดตั้งได้โดยตรง (APK File) เพื่อทดสอบภายในองค์กร จะต้องระบุค่าออปชันใดในโปรไฟล์ Preview ของไฟล์ `eas.json`?

ก. `"buildType": "apk"`

ข. `"android": { "format": "apk" }`

ค. `"android": { "buildType": "apk" }`

ง. `"distribution": "apk"`

ข้อ 8: ชุดไฟล์กำหนดค่าการทำงานใน `eas.json` ทำหน้าที่จัดแจงแบ่งแยกกลุ่มระดับประเภทการ Build เพื่ออะไร?

ก. เพื่อจัดกลุ่มหน้าจอ Route ใน Expo Router

ข. เพื่อแยกคุณลักษณะโปรไฟล์การสร้างแอปพลิเคชัน เช่น สำหรับพัฒนา (Development), สำหรับทดสอบภายใน (Preview) หรือสำหรับจัดส่งสโตร์จริง (Production)

ค. เพื่อจำกัดสิทธิ์ความปลอดภัยการขออนุญาตกล้องและพิกัดแผนที่

ง. เพื่อเก็บรักษาคีย์ anon key และ URL ของฐานข้อมูล

ข้อ 9: ภายหลังจากที่เราทำการสั่งอัปเกรดรหัสแกนหลักของ Expo ในไคลเอนต์สำเร็จ (เช่น ขยับจาก SDK 53 เป็น SDK 54) ข้อใดคือชุดคำสั่งคำสั่งวิเศษของ Expo CLI ที่ต้องรันสั่งการเพื่อแก้ปัญหาความเข้ากันได้ของรุ่นไลบรารีอื่น ๆ (Dependency Reconciliation) ป้องกันแอปพัง?

ก. `npm update --force`

ข. `npx expo install --fix`

ค. `npx expo doctor`

ง. `npx expo upgrade`

ข้อ 10: เมื่อนักพัฒนาเตรียมส่งมอบและอัปโหลดแอปพลิเคชันขึ้นร้านค้า Google Play Store เพื่อเผยแพร่เชิงพาณิชย์เป็นไฟล์รุ่นผลิต (Production) รูปแบบนามสกุลไฟล์แอปประเภทใดที่ Google Play Console บังคับให้จัดส่งอัปโหลดขึ้นสโตร์ในปัจจุบัน?

ก. **.apk** (Android Package)

ข. **.exe** (Executable)

ค. **.aab** (Android App Bundle)

ง. **.ipa** (iOS App Store Package)

เฉลยคำตอบและคำอธิบายโดยละเอียด

เฉลย ข้อ 1: ตอบ ข.

• เหตุผลประกอบ:

• ก. ค. และ ง. ผิด: วิธีเหล่านี้ไม่ใช่สัญญาณกระตุ้นการเปิดหน้าต่าง Dev Menu ของตัวเครื่องเนทีฟ

• ข. ถูก: การเขย่าเครื่องจริง (Shake Device) เป็นกลไกระดับฮาร์ดแวร์เพื่อสั่งเด้งตัวแผงควบคุม Expo Developer Menu ขึ้นมาให้กดเลือกเปิดฟังก์ชันตรวจสอบ

เฉลย ข้อ 2: ตอบ ข.

• เหตุผลประกอบ:

• ก. ค. และ ง. ผิด: Logcat เก็บเพียงรายงาน Log ของระบบ ไม่สามารถปัก Breakpoint หรือใช้งานเครื่องมือ JavaScript Console ได้ และแอปพลิเคชันบราวเซอร์ไม่ควบคุมการทำงานของแอปเนทีฟตรง ๆ

• ข. ถูก: Google Chrome DevTools (สามารถสั่งเปิดรันได้ผ่านคำสั่งดีบั๊กเกอร์ของ Expo CLI คีย์ลัด **j**) ทำหน้าที่เชื่อมช่องทางเข้าเว็บ JS Engine ในแอป และดึงมาสืบค้นและวาง Breakpoints ดับเบิ้ลตรวจสอบได้อย่างราบรื่น

เฉลย ข้อ 3: ตอบ ก.

• เหตุผลประกอบ:

• ข. ค. และ ง. ผิด: คำสะกดอื่นระบุความหมายผิดพลาด

• ก. ถูก: EAS ย่อมาจากคำว่า `**Expo Application Services**` ซึ่งเป็นกลุ่มบริการระบบคลาวด์แบบเช่าคลาวด์ของฝั่ง Expo (ประกอบด้วย EAS Build, EAS Submit และ EAS Update)

เฉลย ข้อ 4: ตอบ ข.

• เหตุผลประกอบ:

• ก. ค. และ ง. ผิด: ไม่เกี่ยวข้องกับขนาดการดึงรูปภาพ ไม่เขียนคำสั่ง CRUD ให้ และไม่ปิดเครือข่ายเพื่อประหยัดแรม

• ข. ถูก: ระบบ EAS Build จะยกภาระงานทั้งหมดไปประมวลผลบนคลาวด์เซิร์ฟเวอร์ของ Expo ทำให้นักพัฒนาที่ใช้คอมพิวเตอร์ Windows หรือ Linux ทั่วไปสามารถกดสั่งเพื่อสร้างแอปพลิเคชันระบบ iOS ออกมาเป็นไฟล์ `.ipa` เพื่อติดตั้งทดสอบได้โดยไม่ต้องซื้อเครื่อง Mac ราคาแพง

เฉลย ข้อ 5: ตอบ ค.

• เหตุผลประกอบ:

• ก. ผิด: `eas.json` คุมการตั้งค่าโปรไฟล์และคอมไพล์ของ EAS CLI

• ข. ผิด: `package.json` เก็บข้อมูลชื่อแพ็คเกจ Node และสคริปต์รันคำสั่ง

• ค. ถูก: `app.json` คือไฟล์ที่เก็บข้อมูลและ Metadata ของโปรเจกต์ Expo ทั้งหมดที่จะติดตัวไปกับโครงสร้างไบนารีหลังคอมไพล์เสร็จ

• ง. ผิด: babel คุมทรานสไพล์ภาษาจาวาสคริปต์

เฉลย ข้อ 6: ตอบ ค.

• เหตุผลประกอบ:

• ก. ข. และ ง. ผิด: สตริงพิมพ์ใหญ่หรือพาธและไอดีตัวเลขไม่ใช่เกณฑ์ยอมรับในมาตรฐานการระบุแอปของ Google และ Apple

• ค. ถูก: มาตรฐานสากลสำหรับชื่อแอปพลิเคชันเพื่อป้องกันการชนกันของชื่อ (Unique Identifier) คือเขียนในรูป Reverse Domain Name เช่น `com.`ชื่อบริษัท.ชื่อแอป เสมอ

เฉลย ข้อ 7: ตอบ ค.

• เหตุผลประกอบ:

• ก. ข. และ ง. ผิด: โครงสร้าง JSON ของ `eas.json` ต้องมีคีย์ย่อยควบคุมตามแพลตฟอร์มชัดเจน คำสั่งอื่นกำหนดผิดจุดประสงค์

• ค. ถูก: รูปแบบโครงสร้างใน `eas.json` ต้องกำหนดคีย์ภายในโปรไฟล์เป็น `"android": { "buildType": "apk" }` เพื่อบังคับให้เซิร์ฟเวอร์คลาวด์ของ Expo เปลี่ยนประเภทจาก App Bundle กลายมาเป็นไฟล์ติดตั้งด่วนประเภท `.apk`

เฉลย ข้อ 8: ตอบ ข.

• เหตุผลประกอบ:

• **ก. ค. และ ง. ผิด:** ไม่เกี่ยวข้องกับ Route, ไม่ใช้ตั้งค่า Permissions บล็อกกล้อง และไม่ใช้เก็บคีย์ลับ Supabase

• **ข. ถูก:** ไฟล์ **eas.json** ถูกดีไซน์มาเพื่อจัดสร้างและแยกแยะสภาพแวดล้อมในการคอมไพล์แอปพลิเคชันปลายทาง ว่าจะใช้เพื่อทดสอบเขียนในเครื่อง, ส่งออกไฟล์เทสให้เพื่อนร่วมงาน หรือสร้างเพื่อเตรียมโพสต์ขึ้น App Stores

เฉลย ข้อ 9: ตอบ ข.

• **เหตุผลประกอบ:**

• **ก. ค. และ ง. ผิด:** ชุดคำสั่งอื่นไม่ช่วยปรับปรุงหรือแก้ไขรุ่นความปลอดภัยของ library อื่นให้ตรงกับ SDK โดยออโต้ และบางตัวใช้สำหรับตรวจสอบความขัดข้องระบบเฉย ๆ

• **ข. ถูก:** **`npx expo install --fix`** คือคำสั่งพิเศษเฉพาะกิจของ Expo ในการตรวจเช็ค library ทุกตัวในไฟล์ **package.json** แล้วสั่งแก้ไขอัปเดตรุ่นให้สัมพันธ์และเสถียรปลอดภัยเข้าเกณฑ์รองรับของ SDK รุ่นใหม่ที่พึ่งอัปเกรดมาทันที

เฉลย ข้อ 10: ตอบ ค.

• **เหตุผลประกอบ:**

• **ก. ผิด:** ไฟล์ .**apk** ใช้สำหรับแจกจ่ายทดสอบภายนอกสโตร์เท่านั้น ปัจจุบันไม่รับอัปโหลดในระดับโปรดักชัน

• **ข. ผิด:** .**exe** เป็นไฟล์โปรแกรม Windows

• **ค. ถูก:** ตั้งแต่ปี 2021 เป็นต้นมา Google Play Console บังคับว่าแอปพลิเคชันใหม่ทั้งหมดที่ส่งขึ้นเป็นโปรดักชันบนสโตร์จะต้องนำส่งข้อมูลในรูปแบบไฟล์ **`Android App Bundle (.aab)`** เท่านั้นเพื่อให้ขนาดแอปพลิเคชันมีขนาดเหมาะสมที่สุดเมื่อถูกโหลดลงโทรศัพท์มือถือปลายทาง

• **ง. ผิด:** .**ipa** เป็นรูปแบบติดตั้งของฝั่ง iOS

ปฏิบัติการท้ายบทเรียน: การดีบั๊กแอปพลิเคชันและการจัดทำไฟล์ติดตั้ง APK ผ่านระบบ EAS Build

ปฏิบัติการในบทเรียนสุดท้ายนี้จะมุ่งเน้นกระบวนการปลายน้ำในการจัดเตรียมความพร้อมของแอปพลิเคชันสู่ระดับโปรดักชัน โดยผู้เรียนจะได้ร่วมกันปรับแต่งไฟล์โครงสร้างคุณลักษณะระบบของแอปพลิเคชัน (**app.json**), กำหนดเส้นทางคอมไพล์คลาวด์เพื่อรับไฟล์ติดตั้งด่วน Android Package (**eas.json**), รันการตรวจสอบและเตรียมปรับปรุงความขัดแย้งของแพ็คเกจสำหรับเตรียมตัวอัปเกรด SDK, ออกแบบหน้าจอแดชบอร์ดวินิจฉัยสำหรับกดจำลองบั๊กหาจุดเกิดเหตุด้วยดีบั๊กเกอร์

(Diagnostics & Debugging Client) และรันชุดคำสั่งถอดรหัสบิลด์ตัวแอปขึ้นไปบนระบบคลาวด์ของ Expo EAS Build

1. วัตถุประสงค์การเรียนรู้

1. สามารถแก้ไขไฟล์ `app.json` เพื่อลงทะเบียนชื่อแพ็คเกจที่เป็นเอกลักษณ์ (Android Package Name) และเวอร์ชันอิงตามหลักเกณฑ์สากลได้ถูกต้อง

2. สามารถจัดสรรและกำหนดค่าโปรไฟล์สำหรับประกอบไฟล์ APK ลงใน `eas.json` ได้ถูกต้อง

3. สามารถเรียกใช้ชุดคำสั่ง `npx expo install --fix` และเคลียร์ Metro cache เพื่อรับมือกับการขยับอัปเกรดรุ่น Expo SDK ได้สำเร็จ

4. สามารถใช้งาน Chrome Developer Tools ในการระบุจุดเกิดเหตุของบั๊กประเภท JavaScript Crash ในคอมโพเนนต์ได้สำเร็จ

5. สามารถใช้งาน EAS CLI ในการรันประมวลผลเซฟและส่งโค้ดขึ้นบิลด์บนคลาวด์จนได้รับลิงก์ดาวน์โหลดหรือรหัส QR Code ไฟล์ติดตั้งจริง

2. โจทย์และข้อกำหนดปฏิบัติการ (Requirements)

ผู้เรียนต้องจัดสร้างโปรเจกต์ React Native Expo โดยมีโครงสร้างดังนี้:

```
โครงสร้างไฟล์ปฏิบัติการ:
├── app.json                          => ลงทะเบียนชื่อแอป คลังภาพไอคอน และรหัส com.yourdomain.diaryapp
├── eas.json                          => ปรับแต่ง Preview Profile ให้สั่ง Build สรุปประเภทไฟล์เป็น APK
└── app/
    └── index.tsx                     => หน้าแอปวินิจฉัย (Diagnostics Dashboard) พร้อมปุ่มทดสอบสกัดบั๊ก
```

การตั้งค่าการจำลองบั๊กและวิเคราะห์ (Diagnostics Engine):

1. **หน้าจอวินิจฉัย (Diagnostics UI):** แสดงรายงานสเตตัสคุณสมบัติของแอป เช่น เวอร์ชัน และระบบปฏิบัติการ

2. **ปุ่มดีบั๊กจำลองข้อผิดพลาด (Trigger Simulation Crash):** เมื่อผู้ใช้กดคลิก ปุ่มนี้จะต้องรันฟังก์ชันที่จงใจให้เกิด Runtime Crash (เช่น พยายามเข้าถึงคุณสมบัติของอ็อบเจกต์ที่ไม่มี

อยู่จริงหรือเป็น **undefined**) เพื่อทดลองเปิด Chrome DevTools ตรวจหาสาเหตุการเกิด Stack Trace

3. **ปุ่มทดสอบความเสถียร (Run Normal API Mock):** ปุ่มสำหรับทดสอบการทำงานปกติเพื่อเปรียบเทียบ

3. ขั้นตอนการลงมือปฏิบัติ (Step-by-Step Tutorial)

ขั้นตอนที่ 1: การลงทะเบียนติดตั้งคลังคำสั่ง EAS CLI

สั่งรันคำสั่งติดตั้งตัวเชื่อมต่อในระบบส่วนกลาง (Global Node) ของคอมพิวเตอร์ของคุณ:

```
npm install -g eas-cli
```

จากนั้นสั่งล็อกอินเข้าระบบบัญชีผู้พัฒนาของ Expo:

```
eas login
```

ขั้นตอนที่ 2: ตั้งรหัสโดเมนกลับใน app.json

ไปที่ไฟล์ **app.json** และระบุรหัส package ของฝั่ง Android (และ bundleIdentifier สำหรับ iOS) ให้ถูกต้อง:

```
"android": {
 "package": "com.student.diaryapp",
 "versionCode": 1
}
```

ขั้นตอนที่ 3: ตั้งค่าโปรไฟล์บิลด์ APK ใน eas.json

รันคำสั่ง **eas build:configure** เพื่อให้ระบบสร้างไฟล์ **eas.json** ขึ้นมา จากนั้นให้เข้าแก้ไขในส่วนโปรไฟล์ **"preview"** ให้ระบุการบิลด์เป็นประเภทไฟล์ APK (เนื่องจากสเปกเริ่มต้นของ EAS จะบิลด์เป็นไฟล์ AAB สำหรับขึ้นสโตร์อย่างเดียว ซึ่งไม่สามารถติดตั้งลงมือถือตรง ๆ ได้):

```
"preview": {
 "distribution": "internal",
 "android": {
  "buildType": "apk"
  }
}
```

ขั้นตอนที่ 4: ฝึกทักษะการตรวจสอบความเข้ากันได้และการเคลียร์แคชกรณีอัปเกรด SDK

ก่อนทำการส่งมอบโค้ดขึ้นระบบคลาวด์ หากมีการขยับรุ่นของ SDK หลัก เราต้องดึงไลบรารีอื่น ๆ ที่มีรุ่นขัดแย้งกันใน `package.json` ให้มาเชื่อมโยงกันอย่างมั่นคงด้วยการใช้คำสั่ง:

```
npx expo install --fix
```

และสั่งเคลียร์แคชของตัว Metro Bundler ที่ค้างสะสมไว้จากการรันแอปข้ามรุ่นด้วยการเรียกใช้:

```
npx expo start -c
```

ขั้นตอนที่ 5: การเปิดเครื่องมือส่องความลับดีบั๊กเกอร์ (Chrome DevTools)

สั่งรันแอปพลิเคชันในโหมดพัฒนาปกติ:

```
npx expo start
```

ขณะที่รันโปรแกรม ให้กดปุ่ม `j` ในหน้าจอคอมพิวเตอร์ หรือเปิดเบราว์เซอร์ Chrome ขึ้นมาแล้วพิมพ์ค้นหา Debugger เพื่อเชื่อมประสาน Console เข้ากับโทรศัพท์มือถือในการปัก Breakpoints

ขั้นตอนที่ 6: สั่งบิลด์แอปพลิเคชันบน EAS Cloud

เปิดเทอร์มินัลใหม่ยิงคำสั่งบิลด์ส่งขึ้นเซิร์ฟเวอร์ Expo เพื่อรับลิงก์ดาวน์โหลด APK:

```
eas build --platform android --profile preview
```

4. โค้ดเฉลยฉบับสมบูรณ์ (Complete Solution)

ผู้เรียนสามารถกำหนดค่าไฟล์สำคัญทั้ง 3 ไฟล์ตามซอร์สโค้ดเฉลยฉบับสมบูรณ์ต่อไปนี้:

ไฟล์ที่ 1: รหัสคอนฟิก `app.json`

```
{
 "expo": {
  "name": "SecureDiary",
  "slug": "secure-diary",
  "version": "1.0.0",
  "orientation": "portrait",
  "icon": "./assets/icon.png",
```

```
  "userInterfaceStyle": "dark",
  "splash": {
   "image": "./assets/splash.png",
   "resizeMode": "contain",
   "backgroundColor": "#0f172a"
    },
  "ios": {
   "supportsTablet": true,
   "bundleIdentifier": "com.student.securediary"
    },
  "android": {
   "adaptiveIcon": {
    "foregroundImage": "./assets/adaptive-icon.png",
    "backgroundColor": "#0f172a"
      },
   "package": "com.student.securediary",
   "versionCode": 1
    },
  "web": {
   "favicon": "./assets/favicon.png"
    }
  }
}
```

ไฟล์ที่ 2: รหัสคอนฟิก eas.json

```
{
 "cli": {
  "version": ">= 10.0.0"
  },
 "build": {
  "development": {
   "developmentClient": true,
   "distribution": "internal"
    },
  "preview": {
   "distribution": "internal",
   "android": {
    "buildType": "apk"
   }
    },
  "production": {
   "distribution": "store"
```

```
    }
  },
 "submit": {
  "production": {}
  }
}
```

ไฟล์ที่ 3: โค้ดอินเทอร์เฟซวินิจฉัย app/index.tsx

```
import React, { useState } from 'react';
import {
  StyleSheet,
  Text,
  View,
  TouchableOpacity,
  ScrollView,
  SafeAreaView,
  Platform,
  Alert
} from 'react-native';

interface BugLog {
  id: number;
  time: string;
  type: 'info' | 'error';
  message: string;
}

export default function DiagnosticsDashboard() {
  const [logs, setLogs] = useState<BugLog[]>([]);
  const [triggerCrash, setTriggerCrash] = useState(false);

  const addLog = (type: 'info' | 'error', message: string) => {
    const newLog: BugLog = {
      id: Date.now(),
      time: new Date().toLocaleTimeString(),
      type,
      message
    };
    setLogs((prev) => [newLog, ...prev]);
  };

 // 1. จำลองการยิงเรียกใช้งาน API ปกติ
  const handleNormalOperation = () => {
    addLog('info', 'เรียกใช้งาน API ดึงข้อมูลตามปกติสำเร็จ');
  };

 // 2. จงใจทริกเกอร์ให้เกิด JavaScript Error เพื่อใช้ทดสอบกับ Chrome DevTools
  const handleTriggerMockError = () => {
```

```
    try {
      addLog('info', 'กำลังเตรียมทริกเกอร์ JavaScript Error...');

    // การทำงานที่ผิดพลาด: พยายามเข้าถึงคุณสมบัติจากตัวแปรที่เป็น undefined
      const mockUserData: any = undefined;
      const userProfileName = mockUserData.profile.name; // จุดนี้จะระเบิดและเด้ง
Error ทันที!

      console.log(userProfileName);
    } catch (error: any) {
    // ดักข้อความส่งเข้าดีบั๊กเกอร์แอป
      addLog('error', `ดักจับข้อผิดพลาดสำเร็จ: ${error.message}`);
      console.error('พบข้อบกพร่องในระบบ (ดูรายละเอียดใน Console):', error);
      Alert.alert('ตรวจสอบพบคอขวดข้อผิดพลาด', error.message);
    }
  };

 // 3. จำลองพฤติกรรมการแอปค้างพังเฉียบพลัน (สำหรับจำลอง Native Crash ในดีบั๊กเกอร์)
  const handleSimulateNativeCrash = () => {
    addLog('info', 'สั่งเปิดใช้งานจำลอง Runtime Crash ใน 1 วินาที...');
    setTimeout(() => {
      setTriggerCrash(true); // จะสั่งพ่นตัวแปรคอมโพเนนต์พัง
    }, 1000);
  };

 // เมื่อตัวแปรเป็น true ระบบจะพังการเรนเดอร์ทิ้งจำลอง
  if (triggerCrash) {
    const brokenData: any = null;
    return (
      <View style={styles.center}>
        <Text>{brokenData.nonExistentField.crashApp}</Text>
      </View>
  );
  }

  return (
    <SafeAreaView style={styles.safeArea}>
      <ScrollView contentContainerStyle={styles.container}>

        {/* หัวข้อเรื่อง */}
        <View style={styles.header}>
          <Text style={styles.headerTitle}>⚙️ App Diagnostics</Text>
          <Text style={styles.headerSubtitle}>โมดูลวินิจฉัยและจำลองข้อผิดพลาดเพื่อทดสอบดี
บั๊กเกอร์</Text>
        </View>
```

```
        {/* ข้อมูล Metadata แอป */}
    <View style={styles.metadataCard}>
          <Text style={styles.cardTitle}>📌 ข้อมูลจำเพาะของแอป (App
Metadata)</Text>
          <View style={styles.metaRow}>
            <Text style={styles.metaLabel}>ชื่อแอป (App Name):</Text>
            <Text style={styles.metaValue}>SecureDiary</Text>
          </View>
          <View style={styles.metaRow}>
            <Text style={styles.metaLabel}>รุ่นเวอร์ชัน (Version):</Text>
            <Text style={styles.metaValue}>1.0.0 (android/versionCode:
1)</Text>
        </View>
          <View style={styles.metaRow}>
            <Text style={styles.metaLabel}>รหัสแพ็คเกจ (Package):</Text>
            <Text style={styles.metaValue}>com.student.securediary</Text>
          </View>
          <View style={styles.metaRow}>
            <Text style={styles.metaLabel}>แพลตฟอร์มที่รัน (OS):</Text>
            <Text style={styles.metaValue}>{Platform.OS === 'ios' ? 'iOS'
: 'Android'}</Text>
          </View>
        </View>

        {/* ปุ่มทริกเกอร์ระบบ */}
        <View style={styles.card}>
          <Text style={styles.cardTitle}>🛠️ แผงควบคุมดีบั๊กเกอร์ (Debug
Controls)</Text>
          <Text style={styles.cardDesc}>
      ทดลองเชื่อมต่อ Chrome Developer Tools (คีย์ลัด 'j' ใน CLI) จากนั้นกดปุ่มจำลองด้านล่างเพื่อหัด
ตรวจวิเคราะห์สาเหตุของบั๊ก
          </Text>

          <TouchableOpacity style={styles.btnNormal}
onPress={handleNormalOperation}>
            <Text style={styles.btnText}>🟢 ทริกเกอร์ทำงานปกติ (API
Success)</Text>
          </TouchableOpacity>

      <TouchableOpacity style={styles.btnWarn}
onPress={handleTriggerMockError}>
            <Text style={styles.btnText}>◯ ทริกเกอร์ข้อผิดพลาด (Catchable
Error)</Text>
          </TouchableOpacity>

          <TouchableOpacity style={styles.btnDanger}
onPress={handleSimulateNativeCrash}>
            <Text style={styles.btnText}>🔴 ทริกเกอร์แอปพลิเคชันระเบิด (Fatal Render
```

```
Crash)</Text>
          </TouchableOpacity>
        </View>

        {/* รายงาน Log Realtime บนจอภาพ */}
        <View style={styles.logCard}>
          <View style={styles.logHeaderRow}>
            <Text style={styles.cardTitle}>📜 บันทึกสถานะหน้าจอ (On-Screen
Logs)</Text>
            <TouchableOpacity onPress={() => setLogs([])}>
              <Text style={styles.clearLogsText}>ล้างสเตต</Text>
            </TouchableOpacity>
          </View>

          <View style={styles.logScrollWrapper}>
            {logs.length === 0 ? (
              <Text style={styles.emptyLogsText}>ยังไม่มีความเคลื่อนไหวเกิดขึ้นในดีบั๊กเกอร์
ขณะนี้</Text>
      ):(
              logs.map((log) => (
                <View key={log.id} style={styles.logLine}>
                  <Text style={styles.logTime}>[{log.time}]</Text>
                  <Text style={log.type === 'error' ? styles.logErrorText
: styles.logInfoText}>
                    {log.message}
                  </Text>
                </View>
       ))
      )}
          </View>
        </View>

      </ScrollView>
    </SafeAreaView>
 );
}

const styles = StyleSheet.create({
  safeArea: {
    flex: 1,
    backgroundColor: '#0f172a',
  },
  center: {
    flex: 1,
    justifyContent: 'center',
    alignItems: 'center',
    backgroundColor: '#7f1d1d',
  },
```

```
  container: {
    padding: 20,
  },
  header: {
    alignItems: 'center',
    marginVertical: 12,
  },
  headerTitle: {
    fontSize: 24,
    fontWeight: 'bold',
    color: '#ffffff',
  },
  headerSubtitle: {
    fontSize: 12,
    color: '#64748b',
    textAlign: 'center',
    marginTop: 6,
    lineHeight: 18,
  },
  metadataCard: {
    backgroundColor: '#1e293b',
    borderRadius: 14,
    padding: 16,
    borderWidth: 1,
    borderColor: '#334155',
    marginBottom: 15,
  },
  cardTitle: {
    fontSize: 14,
    fontWeight: 'bold',
    color: '#ffffff',
    marginBottom: 10,
  },
  cardDesc: {
    color: '#94a3b8',
    fontSize: 11,
    lineHeight: 16,
    marginBottom: 12,
  },
  metaRow: {
    flexDirection: 'row',
    justifyContent: 'space-between',
    paddingVertical: 6,
    borderBottomWidth: 1,
    borderBottomColor: '#0f172a',
  },
  metaLabel: {
```

```
    color: '#94a3b8',
    fontSize: 12,
  },
  metaValue: {
    color: '#38bdf8',
    fontSize: 12,
    fontWeight: '600',
    fontFamily: 'monospace',
  },
  card: {
    backgroundColor: '#1e293b',
    borderRadius: 14,
    padding: 16,
    borderWidth: 1,
    borderColor: '#334155',
    marginBottom: 15,
  },
  btnNormal: {
    backgroundColor: '#10b981',
    borderRadius: 8,
    height: 40,
    justifyContent: 'center',
    alignItems: 'center',
    marginBottom: 10,
  },
  btnWarn: {
    backgroundColor: '#f59e0b',
    borderRadius: 8,
    height: 40,
    justifyContent: 'center',
    alignItems: 'center',
    marginBottom: 10,
  },
  btnDanger: {
    backgroundColor: '#ef4444',
    borderRadius: 8,
    height: 40,
    justifyContent: 'center',
    alignItems: 'center',
  },
  btnText: {
    color: '#ffffff',
    fontWeight: 'bold',
    fontSize: 13,
  },
  logCard: {
```

```
    backgroundColor: '#0f172a',
    borderRadius: 14,
    padding: 16,
    borderWidth: 1,
    borderColor: '#334155',
    minHeight: 180,
  },
  logHeaderRow: {
    flexDirection: 'row',
    justifyContent: 'space-between',
    alignItems: 'center',
    marginBottom: 10,
  },
  clearLogsText: {
    color: '#38bdf8',
    fontSize: 12,
    fontWeight: '600',
  },
 logScrollWrapper: {
    maxHeight: 160,
  },
  emptyLogsText: {
    color: '#475569',
    fontSize: 12,
    textAlign: 'center',
    marginTop: 40,
  },
  logLine: {
    flexDirection: 'row',
    marginBottom: 6,
    alignItems: 'flex-start',
  },
  logTime: {
    color: '#64748b',
    fontSize: 11,
    fontFamily: 'monospace',
    marginRight: 6,
  },
  logInfoText: {
    color: '#e2e8f0',
    fontSize: 11,
    flex: 1,
  },
  logErrorText: {
    color: '#f87171',
    fontSize: 11,
```

```
    fontWeight: '600',
    flex: 1,
  },
});
```

เอกสารอ้างอิง

Expo. (2024). *Debugging*. จาก https://docs.expo.dev/versions/v54.0.0/debugging/tools/

Expo. (2024). *EAS Build Overview*. จาก https://docs.expo.dev/versions/v54.0.0/eas-build/introduction/

Expo. (2024). *EAS CLI*. จาก https://docs.expo.dev/versions/v54.0.0/eas/

React Native. (2024). *Debugging*. จาก https://reactnative.dev/docs/debugging